

Обсуждения, статистику и профили авторов этой публикации смотрите по адресу: <https://www.researchgate.net/publication/386120957>

Обзор анализа нефункциональных требований в рамках SDLC

Статья в журнале «Компьютеры» · Ноябрь 2024 г.
DOI: 10.3390/computers13120308

ЦИТАТА
1

ЧИТАЕТ
275

1 автор:



Сирил Донгмо
Университет Южной Африки
14 ПУБЛИКАЦИЙ 58 ЦИТИРОВАНИЙ

СМОТРЕТЬ ПРОФИЛЬ

Статья

Обзор анализа нефункциональных требований в рамках SDLC

Сирил Донгмо



Кафедра компьютерных наук, Школа вычислительной техники, Колледж науки, техники и технологий (CSET),
Научный кампус, Университет Южной Африки (UNISA), Флорида Парк, Рудепурт 1709, Южная
Африка; dongmc@unisa.ac.za

Аннотация: На сегодняшний день, как в академических кругах, так и в промышленности, были предприняты несомненные усилия для содействия разработке функциональных требований (FR) на различных этапах жизненного цикла разработки программного обеспечения (SDLC). Под функциональными требованиями понимаются потребности пользователей, относящиеся к услугам, которые должна предоставлять программная система. Например, полуформальные или графически основанные подходы, такие как UML, и математически основанные или формальные подходы, такие как Z и связанные с ними инструменты, были разработаны с целью решения FR. В том же духе, большинство предложенных методологий программного обеспечения, например, гибкая разработка программного обеспечения и разработка программного обеспечения на основе моделей, в первую очередь нацелены на функциональные требования. Учитывая важность и даже критичность нефункциональных требований, описывающих качество программных систем и ограничения на них, можно было бы ожидать аналогичного прогресса и в их разработке. Однако, по-видимому, достижение прогресса в области NFR оказалось более сложным из-за сложности требований. В этой связи основной целью данной работы является раскрытие (с академической точки зрения) современного состояния развития НФР посредством обзора публикаций, тщательно отобранных из пяти онлайн-баз данных.

Ключевые слова: нефункциональные требования; анализ NFR; разработка NFR; программная инженерия; SDLC; проектирование требований; систематизированный обзор литературы; SLR



Ссылка: Донгмо, К. Обзор анализа нефункциональных требований в рамках SDLC.

Компьютеры 2024, 13, 308. <https://doi.org/10.3390/computers13120308>

Академический редактор: Паоло Беллависта

Получено: 13 сентября 2024 г.

Пересмотрено: 7 октября 2024 г.

Принято: 9 октября 2024 г.

Опубликовано: 23 ноября 2024 г.



Авторские права: © 2024 принадлежат авторам.

Лицензиат MDPI, Базель, Швейцария.

Эта статья находится в открытом доступе. распространяется на условиях и условия Creative Commons

Лицензия Attribution (CC BY) (<https://creativecommons.org/licenses/by/4.0/>).

1. Введение

Известно, что разработка программного обеспечения представляет собой сложную задачу, включающую различные этапы, каждый из которых требует определённого уровня специальных знаний [1,2]. К настоящему времени разработаны различные методологии, методы и инструменты для дальнейшего развития инженерии программных систем, снижения стоимости и времени производства, особенно для сложных систем, и/или повышения качества конечного программного продукта. Это приводит к появлению двух фундаментальных и взаимодополняющих компонентов инженерии программного обеспечения: функциональных требований (ФТ) и нефункциональных требований (НФТ).

Функциональные требования описывают услуги, которые должно предоставлять программное обеспечение (в процессе разработки), в то время как NFR определяют качество услуги, а также пользовательский интерфейс и ограничения на то, как услуги должны производиться и предоставляться [3,4]. Таким образом, в контрактных документах соглашения об уровне обслуживания параметры качества обслуживания (QoS), также известные как нефункциональные параметры, используются для указания минимального уровня качества обслуживания, который должна гарантировать система. На сегодняшний день основной прогресс в программной инженерии в значительной степени связан с FR. Например, унифицированный язык моделирования (UML), один из популярных полуформальных (графических) подходов к моделированию объектно-ориентированной методологии разработки программного обеспечения, по своей сути не интегрирует механизмы для NFR. То же самое верно и для других методов, таких как agile [5], разработка программного обеспечения на основе моделей (MDSD) [6] и устоявшиеся формальные методы (FM), такие как нотация Z [7]. Тем не менее, в течение последних нескольких десятилетий предпринимались усилия по дальнейшему развитию НФР. Например, методы целеориентированной и

начиная с зарождения структуры NFR [11], произвели революцию в проектировании NFR, объединив анализ FR и NFR в рамках одной и той же (целевой) модели. Также были разработаны подходы, расширяющие существующие методы для FR или объединяющие их для моделирования и анализа NFR. В первую очередь, следует отметить обогащение UML новыми концепциями, использующими профили UML для разработки новых методов, таких как SysML и MARTE [12]. Как и UML, язык моделирования систем (SysML) является стандартом OMG, используемым для поддержки системных инженеров в различных областях, включая производство, аэрокосмическую промышленность, мехатронику, автомобилестроение, энергетику, оборону, безопасность и робототехнику [13]. С другой стороны, MARTE, другой стандартизированный OMG язык моделирования, больше предназначен для моделирования и анализа систем реального времени и встраиваемых систем.

Тем не менее, во многих публикациях, посвященных НФР, по-прежнему часто встречаются утверждения, указывающие на то, что НФР игнорируются, игнорируются или недостаточно разрабатываются из-за их сложности.

Таким образом, цель данной статьи — определить текущее состояние дел в области анализа/разработки нефункциональных требований (NFR), чтобы определить достигнутый прогресс и лучше понять, что ещё предстоит сделать. Основной исследовательский вопрос, на который пытается ответить данная работа, заключается в следующем: каково текущее

состояние разработки нефункциональных требований на протяжении всего жизненного цикла разработки программного обеспечения? (RQ)

Из основного вопроса вытекают два следующих подвопроса: • Какие подходы или механизмы существуют для обработки НФР? (B1) • Можно ли вывести процесс из существующих подходов к НФР, которые охватывают часть или всю SDLC? (RQ2)

Основные выводы данной статьи следующие:

1. Определение текущего уровня анализа и разработки нефункциональных требований в рамках SDLC. Тем самым данная статья проливает свет на пробелы в исследованиях в области анализа нефункциональных требований и, следовательно, предоставляет возможности для разработки будущих исследований, включая возможные темы магистерских и докторских исследований.
2. В этой статье также предпринята попытка раскрыть методы и приемы, которые использовались для разработки различных средств анализа НФР, а также роль, которую такие приемы и методы играют в разработке НФР.
3. В данной работе также были рассмотрены различные этапы жизненного цикла разработки программного обеспечения, для которых были предложены подходы к анализу и разработке НФР. Для каждого этапа жизненного цикла разработки программного обеспечения (SDLC) были определены и обсуждены действия, реализуемые в рамках таких подходов.
4. Эта работа позволила нам выявить ряд будущих работ для дальнейшего исследования в области области анализа и разработки НФР.

После вышеприведенного введения с исследовательскими вопросами, методы исследования, использованные в этой статье, представлены в Разделе 2, включая процесс исследования в Разделе 2.1, строку поиска с выбранными онлайн-базами данных в Разделе 2.2, критерии включения/исключения в Разделе 2.3 и результаты поиска, представленные в Разделе 2.4. Раздел 3 раскрывает и исследует роль существующих методов и методик, используемых в подходах, предложенных для анализа нефункциональных требований. Между тем, текущее состояние разработки NFR в жизненном цикле разработки программного обеспечения (SDLC) обсуждается в Разделе 4, с акцентом на деятельность по инженерии требований в Разделе 4.1, а также на проектирование программного обеспечения в Разделе 4.2 и реализацию в Разделе 4.3. Затем, в Разделе 5, представлены результаты исследования, за которыми следует обсуждение, варианты будущей работы и угрозы валидности в Разделе 6. Наконец, Раздел 7 завершает статью и за ним следует список литературы.

2. Метод исследования

В соответствии с исследовательским луком, предложенным Сондерсом и др. [14], который проиллюстрирован на рисунке 1 и распакован от внешнего слоя к внутреннему и слева направо, это

Работа в большей степени соответствует интерпретациям исследовательской философии благодаря качественному анализу тщательно отобранных научных публикаций. Исследовательский подход основан на индуктивном монометоде, где основным методом исследования является качественный анализ. В данной статье проводится систематизированный обзор литературы для выявления публикаций, из которых извлекаются и качественно анализируются соответствующие данные для поиска ответов на исследовательские вопросы.

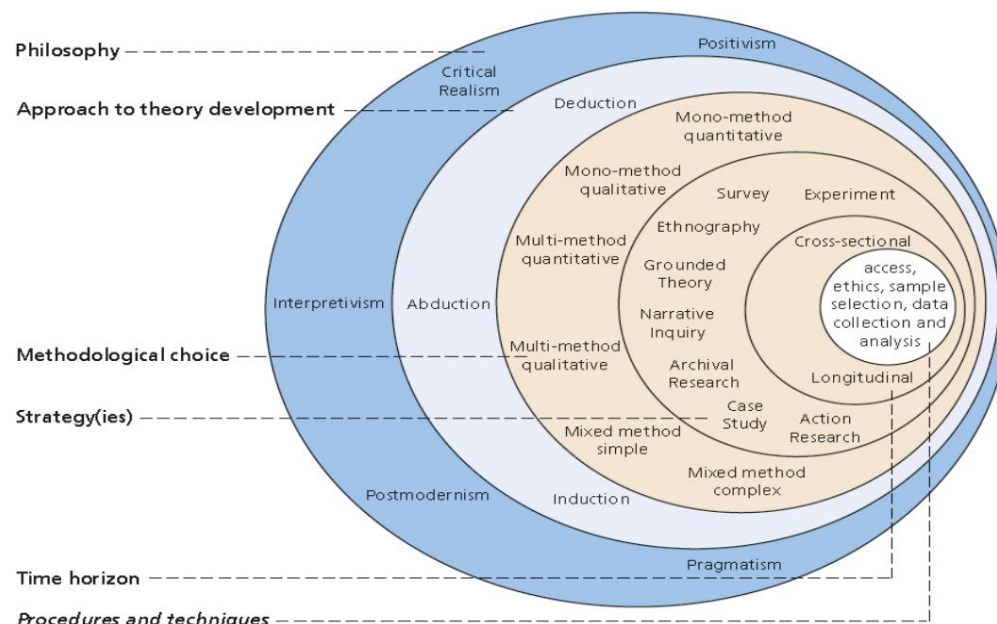


Рисунок 1. Исследовательский лук [14].

Данные, использованные в данной работе, были получены из тщательно отобранных журнальных статей и материалов конференций, посвящённых нефункциональным требованиям и разработке программного обеспечения. Поэтому обзор литературы является важной частью исследовательского процесса. Рассматриваемые документы – это документы, опубликованные или доступные в указанный период, поскольку данная работа представляет собой поперечный анализ.

2.1. Процесс исследования

Чтобы ответить на исследовательские вопросы, был проведен систематизированный поиск литературы публикаций в пяти выбранных онлайн-базах данных (как указано в Таблице 1). Эти базы данных являются одними из наиболее часто запрашиваемых исследователями в области компьютерных наук и информационных технологий. Поисковая строка, используемая для запроса к базам данных, была сформирована путем объединения ключевых слов, сгенерированных из исследовательских вопросов. Из-за особенностей и того, как каждая база данных интерпретирует запрос, строка была немного изменена, чтобы соответствовать требованиям соответствующих баз данных. В зависимости от результатов испытаний окончательный выбор производился по названию статьи и/или аннотации и/или ее ключевым словам. Выбранные статьи были отфильтрованы с использованием критериев включения/исключения. Остальные публикации, те, которые явно касались разработки нефункциональных требований, были синтезированы для ответа на исследовательские вопросы.

Кроме того, запрос, используемый в каждой онлайн-базе данных, также использовался для запросов к базам данных Google Scholar и Springer Link, чтобы обеспечить полноту поиска. Основная цель — исследовать степень интеграции разработки NFR в традиционный жизненный цикл разработки программного обеспечения (SDLC).

Таблица 1. Список выбранных баз данных.

База данных	Веб-адрес через университетскую библиотеку https://
Цифровая библиотека ACM	0-dl-acm-org.oasis.unisa.ac.za/dl.cfm (дата обращения: 11 апреля 2023 г.) https://0-www-
НаукаПрямой	sciencedirect-com.oasis.unisa.ac.za/ (дата обращения: 11 апреля 2023 г.) https://0-www-scopus-
Скопус	com.oasis.unisa.ac.za/ (дата обращения: 11 апреля 2023 г.) https://0-ieeeexplore-ieee-
IEEE Xplore	org.oasis.unisa.ac.za/Xplore/home.jsp (дата обращения: 11 апреля 2023 г.) https://0-webofknowledge-
Веб науки	com.oasis.unisa.ac.za/WOS (дата обращения: 11 апреля 2023 г.)

2.2. Строка поиска и онлайн-базы данных

После ряда испытаний для не-требований были сохранены два ключевых слова : «Нефункциональные требования» и «NFR» . Изначально в список были добавлены другие ключевые слова, такие как «атрибуты качества» , «требования к качеству» и «нефункциональные свойства» , но они не дали положительного результата на этапе испытаний. Второй набор ключевых слов — «спецификация» , «анализ» и «моделирование» . Эти два набора ключевых слов были объединены для формирования следующей общей поисковой строки:

(«нефункциональное требование*» ИЛИ NFR*) И (спецификация ИЛИ анализ ИЛИ модель*).

Как указывалось ранее, эта строка адаптировалась в зависимости от базы данных . Базы данных были выбраны среди тех, которые чаще всего встречаются в обзорах литературы по компьютерным наукам и информационным системам. Список выбранных баз данных представлен в таблице 1.

Доступ к базам данных осуществлялся через онлайн-библиотеку Университета Южной Африки , использованные ссылки указаны во втором столбце Таблицы 1.

2.3 Критерии включения и исключения

В связи с ограничениями по времени для данного исследования были включены только статьи, опубликованные в период с января 2013 года по апрель 2023 года. Хотя этот временной диапазон мог повлиять на количество релевантных работ, окончательный список из 34 релевантных работ был признан приемлемым для целей данного исследования. Что ещё более важно, для исследования были отобраны только публикации, предлагающие руководства, фреймворки, модели и/или любые другие подходы к разработке или оценке НФР в области разработки программного обеспечения или к интеграции анализа НФР в существующие методы или методики. Были исключены постеры и публикации на других языках, а также публикации, не имеющие отношения к анализу НФР. Результаты поиска и процесс отбора включенных публикаций кратко изложены в следующем разделе.

2.4 Результаты поиска

Во-первых, был выполнен автоматизированный поиск по выбранным базам данных по информатике и программной инженерии . Во-вторых, для обеспечения полноты первого поиска использовались базы данных Google Scholar и Springer Link . Количество публикаций, полученных из каждой базы данных, а также дата и время поиска представлены в таблице 2. В ходе первого поиска было извлечено 270 публикаций.

Таблица 2. Результаты поиска по базе данных.

База данных	Дата поиска	Публикации
Цифровая библиотека ACM	11 апреля 2023 г., 12:09:14	48
НаукаПрямой	11 апреля 2023 г., 12:30:15	85
Скопус	11 апреля 2023 г., 14:46:57	51
IEEE Xplore	11 апреля 2023 г., 20:35:21	41
Веб науки	11 апреля 2023 г., 21:47:33	45
Всего публикаций		270

Критерии включения и исключения постепенно применялись для фильтрации первоначального списка публикаций. Для каждого поиска был создан документ bibtex с результатом поиска, и экспортируется в указанную папку в Mendeley Desktop версии 1.19.8. Как показано на рисунке Рисунок 2. Процесс фильтрации был выполнен в Mendeley, начиная с отбора на основе названий статей и аннотаций, что привело к сокращению первоначального списка публикаций до 67. После этого проводилась проверка содержания каждой статьи, с акцент на методологии, обсуждении и выводах, которые привели к исключению 33 Публикации. Таким образом, для исследования было отобрано 34 публикации. Это включает: 21 (что составляет 61,8%) исследование, опубликованное в материалах конференций, 13 (38,2%) из 34) из которых были журнальными статьями.

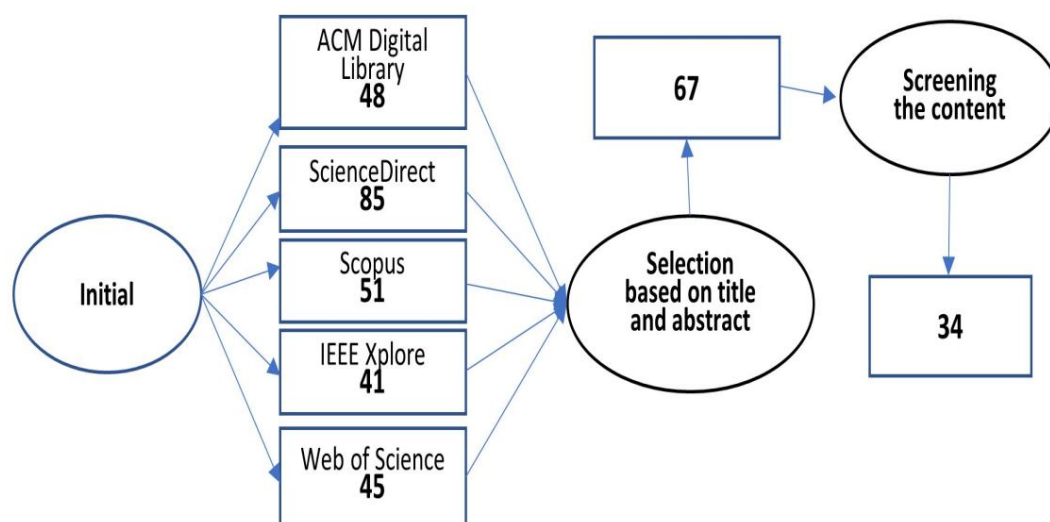


Рисунок 2. Процесс отбора.

Количество публикаций в год и по типу публикации показано на рисунке 3. Синяя линия представляет количество статей, опубликованных в материалах конференций, и красная линия представляет журнальные статьи.

Большинство докладов конференции (всего 8, что составляет 38,1% от всех докладов конференции) были опубликованы в 2013 году. Остальные 13 были опубликованы с 2014 по 2015 год. 2020 г. с частотой около 1 или 2 статей в год, без каких-либо публикаций с 2021 по 2023 г. С 2013 по 2022 год (исключая 2016 и 2017 годы) количество журнальных статей, опубликованных в год, составит приблизительно 0 или 1. Наибольшее число, 5, составляет около 38% от общего числа. статей пришлось на 2023 год, за ним следуют по 2 статьи в год в 2016 и 2017 годах.

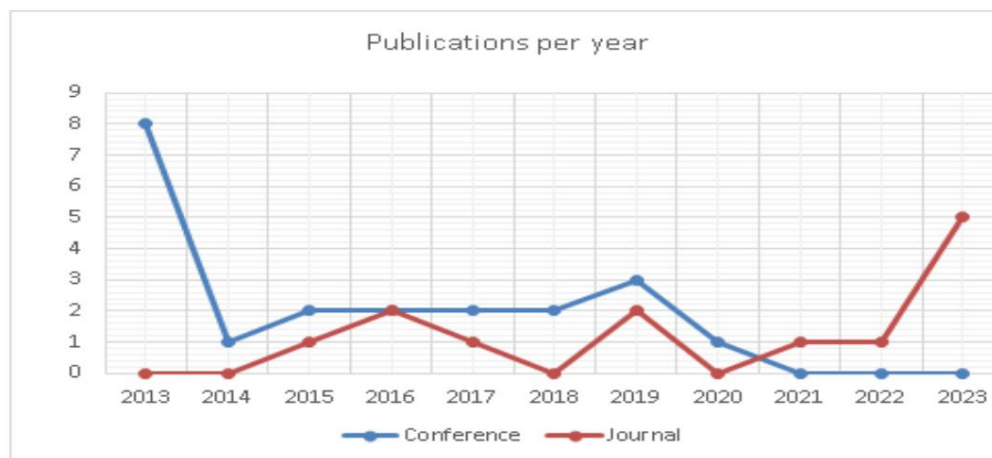


Рисунок 3. Публикации по годам с 2013 по 2023 год.

Важно отметить, что изучение выбранных публикаций показало, что методы и приемы, которые в настоящее время используются в программной инженерии, часто используются или принимаются для анализа. К ним относятся, в частности, UML, XML, алгоритмы машинного обучения/глубокого обучения, математически обоснованные подходы к разработке программного обеспечения и т. д. Таким образом, в следующем разделе более подробно описывается, как используются эти методы и какие аспекты NFR развития, к которым они обращаются.

3. Методы и приемы, используемые при анализе нефункциональных требований (НФТ)

Выбранные подходы к разработке программного обеспечения, используемые или принятые для разработки НФР включают методы GORE, унифицированный язык моделирования (UML), математически подходы, основанные на [15], расширяемый язык разметки (XML) [16] и машинное обучение. Алгоритмы и доступные наборы данных также включены. Диаграмма на рисунке 4 показывает количество публикаций по каждой методике.

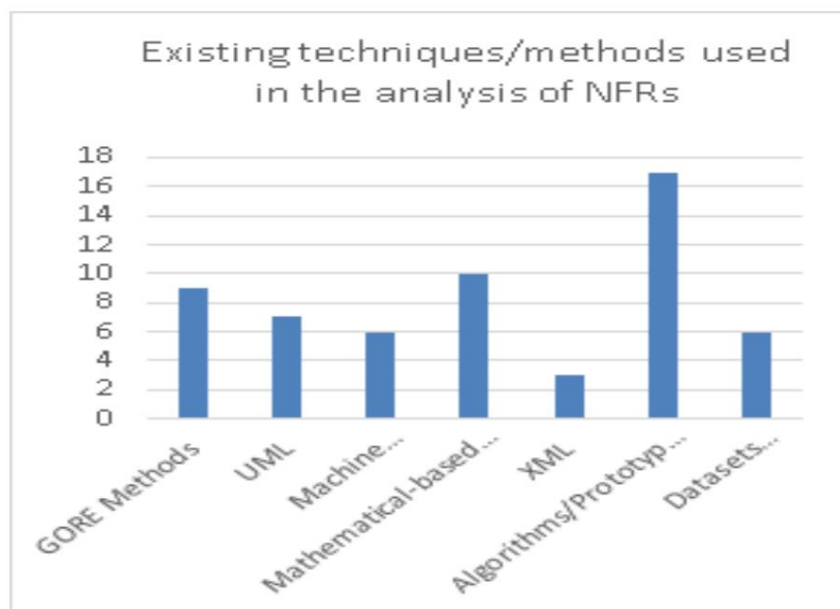


Рисунок 4. Статистика по технологиям, повторно используемым при разработке НФР.

3.1. Методы GORE

Методы целеориентированной инженерии требований (GORE) представляют собой революционные фреймворки инженерии требований, которые объединяют анализ целей, описывающих функциональные требования, и неявных целей, описывающих нефункциональные требования.

включают, среди прочего, фреймворк нефункциональных требований (NFR) [11], фреймворк получения знаний в автоматизированном пространстве (KAOS) [18], фреймворк istar (или i*) [19], Tropos/Secure-Tropos [20,21], язык целеориентированных требований (GRL) [22] и т. д. Центральной концепцией является граф взаимозависимости мягких целей (SIG). SIG инкапсулирует как цели, так и уточнение мягких целей и их эксплуатацию. Уточнение разлагает намеренные элементы (цель, мягкая цель и т. д.) на более подробные, используя концепцию декомпозиции И/ИЛИ. В то время как операция предоставляет средства для удовлетворения целей и мягких целей, она также использует концепцию связей для установления отношений между различными элементами в графе, помогая моделировать и распространять влияние данного решения на другие намеренные элементы (цель, мягкая цель, актор и т. д.). Другим важным аспектом является возможность применения алгоритма оценки (или распространения) к SIG для определения уровня удовлетворения (мягкой) цели для заданной стратегии.

Как показано на диаграмме на рисунке 4, 9 из 34 выбранных для этого исследования публикаций (26,5%) использовали метод GORE при обработке НФР, большинство из которых либо рассматривают модель SIG как прямые входные данные для предлагаемого ими метода, либо сначала преобразуют или расширяют ее новыми концепциями перед ее использованием.

В этой связи следует отметить, что расширение структуры NFR в направленный граф оптимизирует работу с мягкими целями для выбора лучшей стратегии реализации [23].

DeVries и др. [24] предложили трехэтапный инструмент анализа NFR, а именно SOTTER, который вводит новые концепции для смягчения мягких целей в существующую модель целей (KAOS) для облегчения обнаружения нарушения мягких целей и их смягчения. Goncalves и Krishna в [25] предложили расширение модели структуры NFR с взвешиванием мягких целей и их операций для облегчения генерации программных агентов в управляемой модели разработки агентов, выполняемой в динамической среде. Arif и др. в [26] предложили метод для разработки FR, смоделированных с помощью UML, NFR и структуры NFR, где они применили нечеткую логику к модели SIG, чтобы учесть изначально неопределенную и неточную природу связей SIG. Напротив, Mehta и др. в [27] продолжили анализ связей зависимости в модели SIG для улучшения эффективного выбора стратегии для удовлетворения мягких целей. На основе 14 возможностей моделирования конфиденциальности, предложенных Peixoto и др. [28] эти авторы продемонстрировали, что, сравнивая i*, фреймворк NFR и Secure-Tropos, эти языки не полностью поддерживают моделирование требований конфиденциальности. Наконец, Дермеваль и др. в [29] заимствовали концепции фреймворка NFR, чтобы предложить фреймворк для документирования архитектурных решений.

Сравнивая 4 выбранных метода GORE, использующих предлагаемые функции, необходимые для спецификации систем, критически важных для безопасности (SCS), Вилела и др. [30] продемонстрировали, что методы GORE не все одинаково богаты концепциями, необходимыми для спецификации некоторых конкретных систем (например, систем, критически важных для безопасности). Хотя в [28] рекомендуется использовать метод GORE для спецификации требований к конфиденциальности, концептуализация 14 функций (для метамодели), необходимых для спецификации таких требований, выполняется с помощью модели классов UML. Аналогичным образом, в [29] авторы используют диаграмму классов UML для концептуализации метамодели для документирования решений по архитектурному проектированию на основе модели i* для NFR.

3.2. Использование UML-моделирования

для НФР. Унифицированный язык моделирования (UML) [31,32] — это стандартизированный язык моделирования программного обеспечения, который приобрел значительную популярность за последние несколько десятилетий. Несмотря на богатство концепций, относящихся к моделированию и проектированию функциональных возможностей систем, он по своей сути не решает НФР. Однако профили UML [33] позволяют расширять существующие модели стереотипами, которые широко использовались, среди прочего, для поддержки НФР.

Как показано на рисунке 4, 7 из 34 выбранных публикаций (20,6%) использовали UML в предлагаемом подходе к анализу НФР. В их число входят публикации, в которых используются модели EMF Ecore или метамодели. Такие модели можно считать подмоделями или уточненными моделями UML.

В [34] авторы отдали предпочтение использованию профиля UML для моделирования НФР, к которым они применили нечёткую логику для анализа компромиссов НФР, касающихся оптимального удовлетворения НФР. С другой стороны, в [26] предлагается альтернативный подход, заключающийся в связывании НФР из фреймворка НФР с диаграммами прецедентов UML, моделирующими ФР. В том же духе Каур и Шарма в [35] считают целесообразным связывать НФР с прецедентами, описывающими функциональные возможности системы, и таким образом расширять диаграммы прецедентов UML для интеграции НФР. Мартинес и др. [36] предложили подход к обнаружению конфликтов и зависимостей в NFR с использованием сценариев UML и вариантов использования, описывающих функциональные возможности системы. В [37] Халик и др. предложили метамодели EMF Ecore для генерации NFR домена из NFR линейки продуктов с использованием подхода преобразования MDSD. Хотя в [28] диаграмма классов UML не использовалась для представления или анализа NFR, она использовалась для моделирования концептуальной основы конфиденциальности, чтобы обеспечить сравнение между i*, фреймворком NFR и Secure-Tropos для определения, какой из трёх больше подходит для моделирования требований конфиденциальности. Аналогичным образом, в [29] предлагается метамодель UML для документирования решений по архитектурному проектированию, относящихся к модели i* NFR.

3.3 Использование методов машинного обучения (МО) для анализа НФР

Что касается машинного обучения, в данном исследовании были рассмотрены все статьи, использующие алгоритмы машинного обучения (МО) [38], алгоритмы глубокого обучения (ГО) [39] и/или любые инструменты обработки естественного языка (НЛП). Преимущество этих алгоритмов заключается в том, что они позволяют компьютерам обучаться на основе данных, не будучи явно запрограммированными для более точного прогнозирования или принятия более точных решений.

Глубокое обучение (ГО) (подмножество МО), основанное на традиционных нейронных сетях, превзошло традиционное МО, основанное на статистических моделях [40] в различных приложениях, включая задачи обработки естественного языка: извлечение информации из больших документов, классификация текстов и т. д.

Одной из проблем анализа NFR является извлечение таких требований из часто большого документа спецификации требований, написанного на естественном языке, и классификация их по различным категориям. Чтобы преодолеть интенсивные ручные усилия и присущую ему высокую вероятность ошибок, было запрошено использование алгоритмов ML/DL. Рашван и др. [41] предложили онтологическую аннотацию типов NFR и алгоритмы ML, известные как классификатор машины опорных векторов (SVM), для автоматической категоризации предложений NFR по их онтологическим классам. В [42] предлагается улучшенный подход DL, а именно BERT-CNN, для извлечения и классификации NFR. Аналогичным образом, в [43] для классификации NFR запрашивается рекуррентная нейронная сеть (RNN). С другой стороны, Яхья и др. [44] предпочли гибридную модель глубокого обучения, которая объединяет модель RNN и 2 LSML для идентификации и классификации NFR, запрошенных из комментариев пользователей для мобильных приложений. Алашка в [45] решил изучить общие черты, сопоставления и взаимосвязи между нефункциональными требованиями, проанализировав результаты 5 алгоритмов машинного обучения, применённых к существующему набору данных о требованиях. 5 моделей машинного обучения включают в себя метод k-ближайших соседей (KNN), логистическую регрессию (LR), опорную векторную машину (SVM), мультиномиальный наивный байесовский алгоритм (MNB) и случайный лес (RF). Наконец, Сингх и др. [46] предложили систему на основе правил для автоматизированной классификации нефункциональных требований из спецификаций требований с использованием различных инструментов обработки естественного языка.

Таким образом, в 6 из 34 публикаций (17,6%), выбранных для данного исследования, предлагается использовать алгоритмы машинного обучения и инструменты обработки естественного языка для извлечения и классификации ненормативных требований из документов с требованиями.

3.4 Использование математических методов для анализа НФР

Хотя алгоритмы МО по сути основаны на математических моделях, они рассматриваются отдельно, поскольку представляют собой устоявшиеся методы в программной инженерии. В рамках математически обоснованных методов были включены все статьи, включающие математические концепции, отличные от концепций МО/ГО, для анализа НФР. Среди прочего, эти модели или формулы основаны на онтологии, нечеткой логике, теории множеств, формальной логике и т. д. Эти подходы в основном используются для анализа компромиссов конфликтующих НФР, их приоритизации и выбора стратегий, которые наилучшим образом реализуют или удовлетворяют нестрогим целям, описывающим НФР.

Как показано на графике на рисунке 4, 10 из 34 (29,4%) выбранных публикаций использовали некоторую математику в своих методах. Для управления конфликтами между NFR в [47] предлагается фреймворк, основанный на модели онтологии, а именно *sureCM*, для указания требований системы и параметров, необходимых для характеристики конфликтов и упрощения принятия решений. В [23] Аффлек и др. предложили расширение SIG фреймворка NFR в направленный граф, чтобы позволить применять операции теории множеств и арифметические формулы для вычислений для выбора операций, оптимизирующих удовлетворение NFR. Рашван и др. [41] построили онтологическую модель NFR, к которой они применили алгоритмы классификатора ML для классификации NFR. Саадатманд и Тахвили [34] применили нечеткую логику к модели NFR, построенной с профилем UML для анализа компромиссов, чтобы помочь выявить различные альтернативы дизайна, которые приводят к более высокой общей удовлетворенности NFR в системе. Напротив, Ариф и др. [26] предложили расширение модели фреймворка NFR с помощью нечетких понятий для решения проблемы неопределенности и неточности связей вклада, тем самым облегчая анализ влияния операции на другие NFR. Аналогичным образом, Чжан и Ван [48] используют теорию нечетких множеств для проведения анализа компромиссов конфликтующих NFR. Паукар и Бенкомо [49] предложили математическую вероятностную структуру, основанную на моделях частично наблюдаемых марковских процессов принятия решений (POMDP), для принятия решений во время выполнения для удовлетворения NFR. Чтобы облегчить выявление конфликтующих NFR и их приоритезацию, Шах и др. [50] также использовали онтологическую модель для NFR. В том же духе Лю [51] предложил инструмент обнаружения конфликтов, основанный на онтологии эволюции НФР. Наконец, Луангвирия и Конгкандра [52] применили экзистенциальную логику и правила логического вывода к концептуальным графам, представляющим фреймы НФР, для автоматизации обнаружения конфликтующих НФР.

3.5 Роль расширяемого языка разметки (XML) в разработке NFR

Всего 3 (около 9%) из 34 выбранных публикаций использовали XML для анализа NFR. Алмейда и др. [53] предложили XML-спецификацию модели функций линейки программных продуктов, чтобы предложить алгоритм принятия решений для определения наилучшей доступной конфигурации приложения (BSolution) на основе заданной спецификации NFR, которая наилучшим образом удовлетворяет NFR, в то время как Шах и Патель [54] использовали XML-спецификацию схем описания требований, направленную на облегчение переносимости, автоматизации, хранения и прослеживаемости требований. Наконец, в [52] XML-модель NFR, извлеченная из документа спецификации требований и представляющая собой построение концептуальных графов для NFR, используется для обнаружения конфликтов в NFR.

3.6. Алгоритмы, прототипы и моделирование в разработке НФР. В этой категории

рассматривались публикации, в которых алгоритмы, прототипы, моделирование или существующие программные инструменты были включены в предлагаемый подход к анализу НФР. Всего было выявлено 17 (50%) из 34 публикаций, выбранных для данного исследования.

В этих публикациях в основном рассматриваются подходы машинного обучения или математически обоснованные решения для обработки НФР. В целом, алгоритмы предлагаются как средства обработки предлагаемых моделей НФР или реализации заданного метода.

Прототипы и инструменты используются либо для иллюстрации, либо для тестирования.

3.7 Роль существующих наборов данных: Promise и Concordia RE Corpus в разработке NFRs В этом

исследовании были обнаружены два набора данных: наборы данных PROMISE и Concordia. Корпус PROMISE [55] представляет собой репозиторий исследовательских наборов данных, содержащий в общей сложности 326 НФР и 358 ФР. Корпус Concordia RE был создан [41] для их работы. Он содержит 3064 предложения с требованиями, вручную отсортированных по четырём основным классам: ФР, внешнее и внутреннее качество, ограничения и другие НФР. Эти наборы данных в основном используются для тестирования алгоритмов МО/ГО, применяемых для анализа НФР, особенно для извлечения и классификации НФР из больших системных репрезентативных источников (SRS). Тестирование проводится для определения производительности модели МО с точки зрения точности, полноты и F1-оценки, которые обеспечивают количественную оценку предсказательных возможностей модели [56].

Хотя большинство подходов NFR, предложенных в выбранных публикациях, не ориентированы на какую-либо конкретную известную методологию разработки программного обеспечения, некоторые из них, которые обсуждаются в следующем разделе, на нее ориентированы.

3.8. Анализ NFR для конкретных методологий разработки программного

обеспечения. В 12 из 34 публикаций (35,1%), выбранных для данной работы, подходы, предложенные для анализа NFR, специфичны для определенной парадигмы или методологии разработки программного обеспечения. К ним относятся гибкая методология [57–60], разработка на основе моделей (MDD) [25,34,37,61], аспектно-ориентированная разработка программного обеспечения (AOSD) [35], линейка программных продуктов (SPL) [37], самоадаптирующиеся системы (SAS) [49] и сервисно-ориентированная архитектура (SOA) [62]. На рисунке 5 показаны процентные доли 12 публикаций, в которых была выбрана соответствующая методология.

Как показано на рисунке 5, методы Agile и MDD были использованы в 4 публикациях соответственно, что составляет 34% из 12. Эти две методологии изначально не интегрируют механизмы разработки НФР в свои процессы, и, несмотря на их популярность, для полного внедрения НФР необходимы дополнительные исследования. Каждый из четырех оставшихся подходов рассматривался только в 1 публикации из 12 (8%).

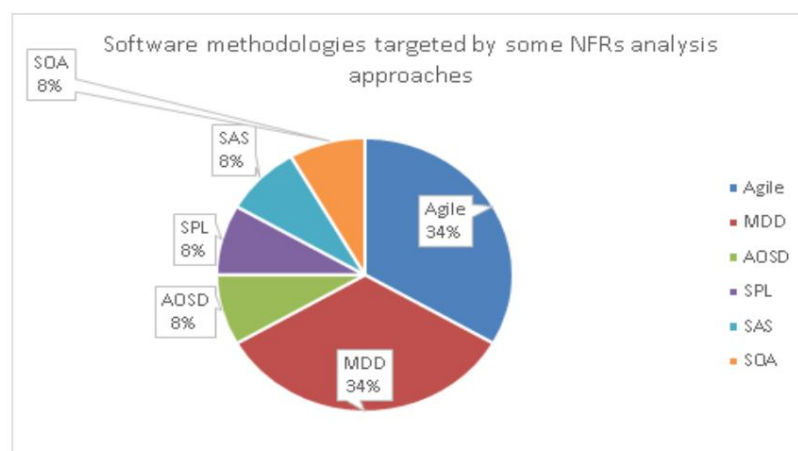


Рисунок 5. Статистика по методологиям, используемым в подходах к анализу НФР.

4. Анализ NFR в SDLC в целом, авторы

выделяют четыре основные фазы для SDLC, хотя для их описания они используют разную терминологию [1,63,64]. Поскольку эта работа направлена на исследование прогресса, достигнутого в разработке NFR в рамках SDLC, здесь рассматриваются только те фазы SDLC, для которых мероприятия были рассмотрены в выбранных публикациях. Для каждой фазы рассматриваются только мероприятия, описанные в выбранных публикациях. Таким образом, определены только пять мероприятий для фазы разработки требований (RE) и только два для фазы проектирования программного обеспечения (SD). Мероприятиями для фазы RE являются выявление NFR, первичный анализ NFR, спецификация и моделирование NFR, анализ конфликтов и компромиссов NFR и выбор стратегий NFR. Двумя мероприятиями SD являются проектные решения/трансформация и документирование/прослеживаемость. Ни одно мероприятие не может быть связано с этапом внедрения программного обеспечения.

На рисунке 6 представлена статистика количества публикаций по видам деятельности, связанным с НФР. Видно, что, как правило, каждая публикация охватывает более одного вида деятельности, не обязательно относящегося к одному и тому же этапу разработки НФР, хотя, по-видимому, большая часть усилий исследователей была сосредоточена на этапе разработки требований. 32 публикации (94% из 34) были посвящены этапу разработки требований, в то время как только 4 (менее 12%) были посвящены этапу проектирования программного обеспечения [29,35,37,53]. Две из этих публикаций обсуждали деятельность как на этапах разработки требований, так и на этапах разработки программного обеспечения [37,53].

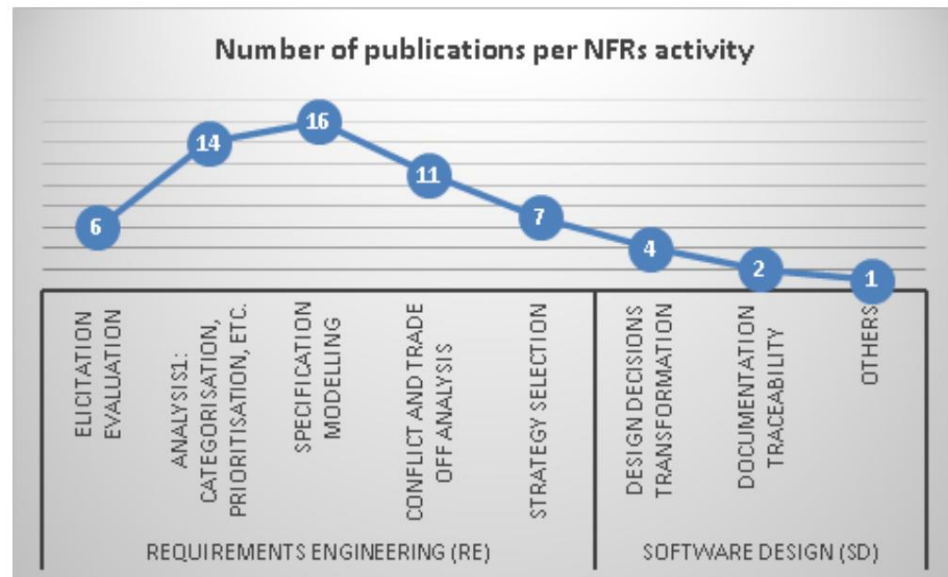


Рисунок 6. Статистика публикаций по видам деятельности НФР.

4.1. Инженерия требований NFR На рисунке

6 видно, что спецификация и моделирование NFR являются наиболее популярной деятельностью, рассмотренной в 16 публикациях, за которыми следуют, соответственно, категоризация и приоритизация требований (14), обнаружение конфликтов NFR и анализ компромиссов (11), выбор лучшей стратегии для удовлетворения NFR (7) и выявление и оценка требований (6).

4.1.1. Выявление и оценка НФР

Выявление требований заключается для инженера-программиста в сотрудничестве с другими заинтересованными сторонами в выявлении и сборе истинных потребностей пользователей путем выявления, поиска или изучения всех возможных источников требований. Это первый и единственный из наиболее сложных видов деятельности по выявлению требований, целью которого является создание надежной спецификации требований к программному обеспечению (SRS) [65]. На этом этапе сбора требований, как правило, не всегда возможно различить функциональные и нефункциональные требования, и, следовательно, эта деятельность является общей для обоих типов требований. Это может быть одной из причин, по которой в настоящем исследовании не удалось найти много публикаций по выявлению требований NFR, поскольку основное внимание уделялось только NFR. Однако в отрасли недостаток выявления NFR также отмечался Вернером и соавторами [66], которые утверждали:

Хотя наше исследование не наблюдало непосредственно за возникновением НФР в трех организациях, выявленные нами практики представляли собой конкретные действия, которые эти организации предпринимали для поддержки реализации и проверки НФР [66].

Из шести статей, посвященных выявлению требований, две из них не фокусировались на выявлении требований [26,30], но были причислены к шести, потому что они использовали выявление требований в качестве промежуточного шага в своем подходе. Остальные четыре были сосредоточены на Agile: Brataas и др. [60] предложили метод выявления требований к масштабируемости в Agile. В то время как Meridji и др. [62] разработали структуру для выявления требований безопасности, Amorndettawin и Senivongse [59] предложили шаблоны/шаблоны для выявления и представления NFR, а Aljallabi и Mansour [57] предложили шестиступенчатый процесс выявления и анализа NFR в Agile на основе двух существующих подходов [67,68]. Относительная концентрация усилий исследователей на Agile может быть оправдана тем фактом, что эта методология популярна, но, естественно, не имеет механизмов для выявления и анализа NFR.

4.1.2. Первичный анализ НФР: категоризация и приоритизация. В зависимости от подхода и

методов, используемых для сбора требований, предварительный анализ и оценка отдельных требований могут быть выполнены уже на этом раннем этапе. Однако, помимо прочего, в целях автоматизации, основная часть анализа возможна только при наличии начального списка требований, как правило, выраженного на естественном языке (например, английском). К ним относятся, помимо прочего, обеспечение корректности, полноты, внутренней и внешней согласованности требований, их классификация и приоритизация, выявление и разрешение конфликтов между ними, выбор оптимальных стратегий для выполнения НФР и т. д.

В этой работе этап анализа разделен на две части, чтобы отделить действия, общие как для FR, так и для NFR, и действия, присущие исключительно NFR. Как показано на рисунке 6, 14 из выбранных 34 публикаций (41%) рассматривали основные аналитические действия [37,41–47,50,52,57,58,69,70], при этом большая часть усилий была направлена на автоматизацию процесса извлечения требований из документа с требованиями (созданный на этапе сбора требований) и их классификацию и/или приоритизацию. Как более подробно обсуждается в разделе 3, авторы предлагают подходы, которые объединяют различные методы, такие как нечеткая логика, онтология, машинное обучение/глубокое обучение, XML и т. д., для автоматизации извлечения NFR из документов с требованиями, а также их классификацию и приоритизацию.

4.1.3. Спецификация NFR и моделирование программного обеспечения

Хотя спецификация требований к программному обеспечению может начинаться уже на этапе выявления, в целом и на практике первоначальный документ с требованиями, составленный на этапе выявления, написан на естественном языке и содержит смесь FR и NFR, а также другую полезную информацию [4,65]. После этого, в зависимости от принятого подхода, требования извлекаются из документа и специфицируются для дальнейшего анализа и разработки. В этом исследовании было отмечено, что за пределами этапа выявления и оценки требований любая важная разработка NFR, предложенная в публикации, предшествует спецификации требований, использующей известный или предложенный метод с соответствующим языком моделирования, таким как варианты использования для UML или фреймворка NFR для метода GORE. По сравнению с другими видами деятельности, это может объяснить большое количество статей, которые включают спецификацию требований в свои предлагаемые подходы к анализу NFR: 16 из выбранных 34 публ

Было отмечено, что некоторые из публикаций, изученных в этой работе, используют более одного метода спецификации требований в предлагаемом подходе к анализу NFR; основными подходами, рассмотренными и обсуждаемыми в Разделе 3, являются методы GORE (фреймворк NFR, KAOS, iStar, GRL и TROPOS) [23–28,30,62], UML [26,28,34–37], Ontology [23,26,34,47,50,51] и XML [52,54].

4.1.4. Второй анализ NFR: обнаружение конфликтов, анализ компромиссов и выбор стратегии Хотя на рисунке 6 они разделены,

обнаружение и разрешение конфликтных отношений между NFR и выбор наилучшего решения для удовлетворения NFR являются тесно связанными видами деятельности, которые нелегко разделить на практике. Они относятся к важным проблемам анализа NFR, для которых были предложены различные подходы, такие как методы, основанные на онтологиях, посредством которых онтологическая модель требований конструируется и используется в качестве входных данных для предлагаемого метода [47,50,51]. При подходе, основанном на цели, целевая модель требований иногда трансформируется или расширяется новыми концепциями, и впоследствии предлагается метод или структура для анализа модели [23–25,27]. В том же духе были также предложены подходы, основанные на нечеткой логике [34,48], а также подходы, основанные на концептуальных графах [52], частично наблюдаемых марковских процессах принятия решений (POMDP) [49] и матрицах [57].

4.2. Ненормативные требования (NFR) в проектировании программного обеспечения

Влияние НФР на решения по проектированию программного обеспечения для создания программного обеспечения надлежащего качества давно признано исследователями [71]. Следовательно, ожидания

заключается в том, чтобы исследователи предлагали процессы проектирования программного обеспечения, которые полностью интегрируют NFR и включают документирование и прослеживаемость проектных решений. В связи с этим 3 из 34 выбранных публикаций (менее 9%) способствовали по крайней мере одному из этих ожиданий. В [29] предлагается метамодель для документирования проектных решений (и, следовательно, связывания проектирования с NFR) и облегчения прослеживаемости. Кхаттер и др. [61] предложили подход к интеграции нефункциональных требований в архитектуру, управляемую моделью. И последнее, но не менее важное: Алмейда и др. [53] разработали подход к принятию решений на основе NFR для выбора конфигурации выполнения, которая наилучшим образом удовлетворяет NFR в динамической среде.

4.3. Нестандартные требования (NFR) при

реализации программного обеспечения. Этап разработки и распространения NFR включает в себя все методы и инструменты, используемые для управления влиянием NFR на различные модели и конечный программный продукт. Ключевым моментом здесь является изучение степени интереса, который этот этап представлял для исследователей (и/или практиков). Предполагается, что этому этапу, как правило, уделялось недостаточно внимания. Ни одной статьи, посвященной NFR на этапе реализации SDLC, не найдено. Это требует дальнейшего изучения как в отрасли, так и в академической среде.

5. Результат исследования

Основной целью данной работы было изучение текущего состояния разработки нефункциональных требований на протяжении всего жизненного цикла разработки программного обеспечения. Для этого был проведен систематизированный обзор 34 публикаций, опубликованных в период с января 2013 года по апрель 2023 года. Все публикации были тщательно отобраны из пяти авторитетных онлайн-баз данных. Основной исследовательский вопрос был разделен на два дополнительных вопроса.

5.1. Какие подходы или механизмы существуют для обработки нефункциональных запросов? (RQ1)

При разработке НФР используются различные методы, методики и инструменты: методы GORE, UML, машинное обучение, глубокое обучение и/или инструменты обработки естественного языка, другие математические методы, такие как нечеткая логика, онтология, теория графов и логика вывода, нотация на основе XML, алгоритмы, прототипы и существующие наборы данных. При использовании таких методов нотации, как методы GORE, UML и XML, авторам иногда приходится расширять исходную методику новыми концепциями, чтобы сделать её более применимой к предлагаемому методу. Например, отмечено преобразование фреймворка NFR в ориентированный граф для облегчения выбора операции, наилучшим образом удовлетворяющей NFR [23]. Другим примером является использование профиля UML для включения нечетких понятий в модель UML и фиксации различных альтернативных признаков [34]. Некоторые авторы также комбинируют различные методы для построения механизма, отвечающего за разработку NFR (как показано на рисунке 6 и обсуждается в разделе 4). Например, Луангвирия и Конгкандра [52] предложили метод обнаружения конфликтов внутри NFR, использующий расширенную версию XML, а именно XML-фреймы NFR для представления NFR, извлеченных из документов с требованиями к программному обеспечению, а также концептуальный граф (математический метод) для структурирования NFR и, следовательно, для облегчения анализа и выявления потенциальных конфликтов.

За последние десять лет усилия исследователей по анализу NFR были сосредоточены на деятельности по проектированию требований и проектированию программного обеспечения, при этом меньшие усилия были направлены на разработку или реализацию программного обеспечения. В RE охватываемые виды деятельности включают выявление и оценку NFR, их классификацию и приоритизацию, спецификацию/моделирование NFR, обнаружение конфликтов и анализ компромиссов, а также выбор наилучшей операции или стратегии для удовлетворения NFR. Напротив, выявление NFR не получило такого большого внимания в выбранных публикациях по сравнению с другими видами деятельности в RE, такими как классификация NFR, приоритизация, спецификация/моделирование, обнаружение конфликтов и анализ компромиссов. В целом, методы, предлагаемые для разработки NFR, используют методы GORE, UML, XML и онтологию для спецификации и моделирования действий, а также функционирования NFR. Методы ML/DL вместе с некоторыми другими инструментами обработки естес

Нечёткая логика, теория графов и логика вывода используются для анализа моделей NFR, связанных с их классификацией, приоритизацией, обнаружением конфликтов и т. д. Некоторые методы NFR предлагаются для конкретной методологии или среды разработки программного обеспечения и могут быть неприменимы в других средах или нуждаться в адаптации. Существуют подходы для гибкой разработки, разработки на основе моделей, аспектно-ориентированной разработки программного обеспечения, линейки программных продуктов, самоадаптирующихся систем и сервисно-ориентированной архитектуры.

5.2. Можно ли вывести процесс из существующих подходов NFR, который охватывал бы весь SDLC или его часть? (RQ2)

Программная инженерия – это непрерывный многоэтапный процесс, начинающийся с выявления требований пользователя и завершающийся тестированием программного продукта. В связи с этим задача в рамках процесса должна принимать в качестве входных данных выходные данные предыдущей деятельности и создавать выходные данные, которые могут быть использованы в других видах деятельности. Таким образом, необходимо исследовать возможность структурирования существующих подходов к разработке НФР в группы, для которых выходные данные одной группы могут быть использованы в качестве входных данных другой группы. Это может потребовать более глубокого качественного анализа каждого из существующих подходов (что выходит за рамки данной работы и представляет собой отдельную тему исследования). Однако для иллюстрации этой идеи мы рассмотрим виды деятельности, связанные с реферированием (RE), на рисунке 6 и методы, предлагаемые для каждого вида деятельности.

Можно рассмотреть следующие группы: группу подходов к выявлению и оценке NFR, группу категоризации и приоритизации NFR, еще одну группу методов или методик для спецификации и моделирования NFR, а также группу предлагаемых подходов для обнаружения конфликтов в NFR, проведения анализа компромиссов и выбора среди альтернативных решений стратегии, которая наилучшим образом удовлетворяет NFR. Только деятельность по выявлению принимает входные данные из различных источников: заинтересованных сторон, бизнес-документов и процессов, бизнес-среды, политик, стандартов и т. д. для создания исходного документа(ов) требований. Эти требования извлекаются из документа, классифицируются и приоритизируются, что приводит к более структурированным и проработанным документам требований, которые необходимо указать, уточнить и использовать. Их также можно использовать для создания других моделей программного обеспечения. С помощью методов GORE FR и NFR из документа(ов) требований указываются, уточняются и используются, что приводит к модели SIG (граф взаимозависимости мягких целей). С помощью UML можно создавать больше моделей, включая диаграммы сценариев, диаграммы вариантов использования, диаграммы классов, диаграммы деятельности и т. д. Те же входные данные можно также задать с помощью XML или создать онтологическую модель. Это показывает, что из одних и тех же входных данных можно получить различные выходные данные в разных форматах. Это не обязательно означает препятствие для последующих действий. Во-первых, потому что различные выходные форматы можно преобразовать в общий формат, например, XML. Во-вторых, каждый выходной файл может быть подходящим входным файлом для одного или нескольких методов, которые

Эта иллюстрация показывает, что действия по разработке NFR (от выявления требований до документирования решений по архитектурному проектированию) могут быть связаны (итеративно) друг с другом через промежуточные модели разрабатываемой системы. Они являются итеративными в том смысле, что на любом этапе разработки, если обнаружена ошибка (например, при верификации и/или валидации промежуточной модели) или если выявлено новое требование, некоторые из предыдущих действий необходимо будет повторить для исправления ошибки или включения нового требования. Следовательно, можно разработать процесс или комплексную структуру для охвата первых двух этапов SDLC. Чтобы иметь полный процесс разработки NFR для всего SDLC, потребуются больше работы по созданию новых методов, методик и инструментов, которые можно использовать для расширения анализа/разработки NFR с этапа проектирования программного обеспечения до внедрения и тестирования конечного программного продукта. Такие исследования могут также включать дальнейшее изучение для выявления дополнительных исследований, которые эта статья не смогла найти, чтобы объединить части для построения полного процесса, охватывающего все этапы SDLC.

6. Обсуждение, будущая работа и угрозы валидности 6.1. Обсуждение

Из обсуждения в разделе 4.1.1 следует, что, несмотря на свою важность и критичность, выявление НФР не было в центре внимания большую часть последнего десятилетия. Это наблюдение может быть оправдано тем фактом, что ранние действия по разработке требований к программному обеспечению являются общими для всех типов требований. На данном этапе автор согласен с Экхардтом и др. [72], которые утверждали, что многие НФР фактически описывают поведенческие свойства и должны рассматриваться аналогично функциональным требованиям. На практике не всегда возможно определить тип требований до того, как они будут проанализированы и уточнены. Поэтому предполагается, что сужение выборки публикаций до сосредоточения исключительно на НФР привело бы к исключению некоторых работ, посвященных выявлению требований в целом. Для достижения более обоснованного вывода, возможно, потребуются дополнительные исследования в этом направлении.

Из обсуждения в разделе 4.2 следует, что менее 9% из 34 выбранных публикаций пытались разработать НФР после фазы RE. Также отмечено, что основной целью разработки НФР на фазе RE является разработка наилучшей стратегии для выполнения НФР. Стратегия – это программное решение, известное в методах GORE как задачи, функции и ресурсы, необходимые для выполнения НФР [22]. Это представление подразумевает, что разработка НФР завершается с получением такой стратегии. Хотя данная работа, по-видимому, соответствует этой идее (только около 9% исследований, посвященных анализу НФР, выходят за рамки фазы RE), она контрастирует с насущной необходимостью параллельной разработки НФР и FR, которая давно стоит на повестке дня [73,74].

В разделе 3 данного исследования было выявлено использование различных методов при разработке НФР, включая, помимо прочего, машинное обучение/глубокое обучение, нечеткую логику, онтологии и т. д. В то же время, устоявшиеся формальные методы (ФМ), такие как Z-нотация [7], Object-Z, VDM, Event B и т. д., не упоминались. ФМ используют математические концепции для описания и обоснования свойств программного обеспечения или системы на ранней стадии разработки [15]. Они подходят для систем, критически важных с точки зрения безопасности и защиты [7,75]. Хотя, как правило, они не интегрируют механизмы для НФР, учитывая их первостепенную задачу — качество и корректность, по сути, от них можно было бы ожидать, что они найдут свой путь к анализу НФР или интегрируют НФР в свои собственные процессы. Это также требует дальнейшего изучения взаимосвязи и интеграции ФМ и НФР.

6.2. Будущая работа

Из данного исследования, особенно из результатов, представленных в Разделе 5, и обсуждения в Разделе 6, следует, что необходимо провести дальнейшее исследование для рассмотрения некоторых важных аспектов разработки НФР, которые не рассматриваются в данной работе.

Первая задача на будущее — создание комплексного процесса анализа и разработки НФР на протяжении всего жизненного цикла разработки программного обеспечения. Процесса, объединяющего сильные стороны существующих методов и инструментов.

Наличие полного процесса и/или методологии для NFR на протяжении всего цикла разработки ПО может позволить разработать новые методы реверс-инжиниринга или реинжиниринга NFR. За последние несколько десятилетий традиционный реверс-инжиниринг программного обеспечения разработал различные способы восстановления и документирования функциональности программного обеспечения, не уделяя должного внимания NFR. Вероятно, это связано с отсутствием отлаженных процессов для NFR в прямой разработке.

Интересным и, безусловно, полезным направлением исследований в области анализа НФР для создания качественных программных продуктов было бы объединение НФР и существующих ФМ, таких как Z/Object-Z. Можно рассмотреть два взаимодополняющих альтернативных подхода: во-первых, интеграция ФМ в разработку НФР, что может привести, например, к использованию программ-доказательств для проверки НФР на ранней стадии разработки. Другой подход заключается в интеграции НФР в процессы ФМ, например, для обеспечения того, чтобы требования к качеству, ожидаемые от конечной программной системы, были внедрены в процесс на ранней стадии и использовались для руководства разработкой на протяжении всего процесса.

В данной исследовательской работе отмечено отсутствие в выбранных публикациях информации об испытаниях и валидации НФР. Это требует дальнейших исследований, которые могли бы включать эмпирические исследования в промышленности и/или более детальные теоретические исследования, посвященные испытаниям и валидации НФР.

Было бы интересно расширить эту работу, создав, например, таксономию подходов NFR, основанную на предмете программной системы или типе создаваемой системы, например, информационные системы, интеллектуальные системы, рекомендуемые системы, и/или исследовать системы, созданные к настоящему времени в отрасли, с целью обоснования применимости/эффективности существующих подходов.

6.3. Угрозы валидности.

Поскольку настоящий обзор литературы подготовлен одним автором, он осознаёт риск предвзятости, особенно при выборе публикаций, извлечении данных из выбранных публикаций, а также при анализе извлечённых данных. Для минимизации этих рисков в статье максимально соблюдались протоколы SLR, а для анализа и упрощения ведения документации использовались электронные таблицы Microsoft Excel (Microsoft® Excel® для Microsoft 365 MSO (версия 2409, сборка 16.0.18025.20030)).

Для повторяемости этой работы включены строка запроса, дата и время запроса каждой выбранной онлайн-базы данных, а также количество записей, созданных для каждой базы данных. в Таблице 2.

7. Выводы

В данной статье представлен систематизированный обзор литературы для изучения текущего состояния разработки НФР. В общей сложности 34 публикации, включая 13 журнальных статей и 21 доклад конференций, опубликованные за последние десять лет, были тщательно отобраны из пяти авторитетных онлайн-баз данных: ACM Digital Library, ScienceDirect, Scopus, IEEE Xplore и Web of Science. Для обеспечения полноты первичного поиска использовались базы данных Google Scholar и Springer Link .

Изучение выбранных публикаций выявило различные методы/методы разработки программного обеспечения и их роли в предлагаемых подходах к анализу и разработке NFR, включая методы GORE, UML, XML и Ontology, в основном для спецификации и моделирования NFR, а также их уточнение и эксплуатацию для некоторых из этих методов. Машинное обучение, алгоритмы глубокого обучения и некоторые другие инструменты обработки естественного языка, нечеткая логика, теория графов, логика вывода и т. д. для извлечения NFR из документов с требованиями, классификации NFR, приоритизации, анализа конфликтов и компромиссов и выбора наилучших стратегий для удовлетворения NFR. Отмечено, что 50% существующих подходов включают алгоритмы для реализации своих методов и/или прототипы для иллюстрации их работы или для тестирования. Подходы, основанные на ML и/или DL, в основном используют для своего тестирования следующие доступные наборы данных: корпус PROMISE [55] и корпус Concordia RE [41].

В данной работе также исследовалось формирование NFR на протяжении всего жизненного цикла разработки программного обеспечения. Как показано на рисунке 6 и обсуждается в разделе 4, большинство существующих подходов к NFR, по-видимому, сосредоточены на этапах разработки требований , уделяя очень мало внимания проектированию и реализации программного обеспечения. В данной статье предполагается, что выявлению NFR в литературе уделено недостаточно внимания. Это может быть объяснено тем, что на раннем этапе разработки программного обеспечения для FR и NFR выполняются одни и те же действия, поэтому выбор публикаций, посвященных исключительно NFR, может быть не совсем целесообразным.

Финансирование: 54% APC финансировалось из фонда платы за страницы статей в аккредитованных журналах Unisa (финансирование из фонда институциональных исследований), а оставшиеся 46% APC оплачивались из фонда стимулирования результатов исследований для кафедры компьютерных наук, школы компьютерных наук, CSET, UNISA.

Заявление о доступности данных: Все данные содержатся в статье.

Благодарности: Я выражаю благодарность моему коллеге Левину Найлу за вычитку и редактирование этой статьи.

Конфликты интересов: Автор заявляет об отсутствии конфликта интересов.

Ссылки

1. Сомервилль, И. Программная инженерия, 8-е изд.; Эддисон-Уэсли, Person Education: Харлоу, Великобритания, 2007.
2. Димеска, К.; Савоска, С. Модель разработки программного обеспечения с использованием методов RAD и стандарта ISO/IEC 12207. В материалах 8-й Международной конференции по прикладным Интернет и информационным технологиям, Санта-Барбара, Калифорния, США, 15–18 октября 2018 г.; Том 8, стр. 48–51.
3. Чунг, Л.; Никсон, Б.А.; Ю, Э.; Милопулос, Дж. Нефункциональные требования в программной инженерии; Springer Science & Business Media: Берлин/Гейдельберг, Германия, 2012; Том 5, № 4. ISO/IEC/IEEE 29148:2018(E); Международный стандарт ISO/IEC/IEEE – Системная и программная инженерия – Процессы жизненного цикла – Разработка требований. IEEE: Пискатауэй, Нью-Джерси, США, 2018; стр. 1–104. <https://doi.org/10.1109/IEEESTD.2018.8559686>.
5. Абрахамссон, П.; Сало, О.; Ронкайнен Дж.; Варста, Дж. Гибкие методы разработки программного обеспечения: обзор и анализ. arXiv 2017, arXiv:1709.08439.
6. Фельтер, М.; Шталь, Т.; Беттин, Дж.; Хаазе, А.; Хельсен, С. Разработка программного обеспечения на основе моделей: технологии, проектирование, управление; John Wiley & Sons: Хобокен, Нью-Джерси, США, 2013.
7. Спайви, Дж. М. Нотация Z: Справочное руководство; Prentice Hall International (UK) Ltd.: Хартфордшир, Великобритания, 1992.
8. Хоркофф Дж.; Айдемир, ФБ; Кардозо, Э.; Ли, Т.; Мате, А.; Пайя, Э.; Сальнитри, М.; Пирас, Л.; Милопулос, Дж.; Джорджини, П. Целеустремленность Инженерные требования: расширенное систематическое картографическое исследование. Requir. Eng. 2019, 24, 133–160.
9. Дальпиаз, Ф.; Франч, Х.; Хоркофф, Дж. Языковой справочник Istar 2.0. arXiv 2016, arXiv:1605.07767.
10. Ван Ламсверде, А. Целеориентированная разработка требований: путеводитель. В материалах пятого международного симпозиума IEEE по разработке требований RE'01, Вашингтон, округ Колумбия, США, 27–31 августа 2001 г.; стр. 249.
11. Чунг, Л.; Никсон, Б.А.; Ю, Э.; Милопулос, Дж.; Чунг, Л.; Никсон, Б.А.; Ю, Э.; Милопулос, Дж. Структура NFR в действии. Нефункциональные требования в программной инженерии; Международная серия по программной инженерии; Springer: Бостон, Массачусетс, США, 2000; Том 5, стр. 15–45.
12. Гомаа, Х. Обзор UML, SysML и MARTE. Разработка программного обеспечения реального времени для встраиваемых систем; Кембриджский университет. Издательство: Кембридж, Великобритания, 2016; стр. 12–31.
13. Вольни, С.; Мазак, А.; Карпелла, К.; Гейст, В.; Виммер, М. Тринадцать лет SysML: систематическое картографическое исследование. Программное обеспечение. Модель. 2020, 19, 111–169. <https://doi.org/10.1007/s10270-019-00735-y>.
14. Сондерс, М.; Бристу, А. Методы исследования для студентов бизнес-факультетов. Предисловие и глава 4, 9-е изд. 2023; Pearson: Нью-Йорк, штат Нью-Йорк, США; Харлоу, Великобритания, 2023; стр. i–xxvii, 128.
15. О'Реган, Г. Математические подходы к качеству программного обеспечения; Springer: Лондон, Великобритания, 2006; стр. I–XV, 1–231.
16. Брей, Т.; Паоли, Дж.; Сперберг-Маккуин, К. М.; Малер, Э.; Йерго, Ф. Расширяемый язык разметки (XML) 1.0; Wiley Publishing: Индианаполис, Индиана, 2008.
17. Донгмо, К.; Ван дер Полл, Дж. А. Улучшенный подход к построению моделей нотации пользовательских требований (URN). Системы 2023, 11, 301.
18. Дарденн, А.; ван Ламсверде, А.; Фикас, С. Целенаправленное получение требований. Sci. Comput. Program. 1993, 20, 3–50. [https://doi.org/10.1016/0167-6423\(93\)90021-G](https://doi.org/10.1016/0167-6423(93)90021-G).
19. Фрэнч, Х.; Лопес, Л.; Керес, К.; Коломер, Д. Структура i* для целеориентированного моделирования; Springer: Берлин/Гейдельберг, Германия, 2016; стр. 485–506. https://doi.org/10.1007/978-3-319-39417-6_22.
20. Брешини, П.; Перини, А.; Джорджини, П.; Джунчилья, Ф.; Милопулос, Дж. Тропос: Методология разработки агентно-ориентированного программного обеспечения. Auton. Agents Multi-Agent Syst. 2004, 8, 203–236.
21. Муратидис, Х.; Джорджини, П. Безопасный Tropos: расширение методологии Tropos, ориентированное на безопасность. Int. J. Software Eng. Knowl. англ. 2007, 17, 285–309. <https://doi.org/10.1142/S0218194007003240>.
22. МСЭ-Т, Рекомендация Z.151 (10/12), Нотация требований пользователя (URN) – Определение языка. Женева, Швейцария, октябрь 2012 г. Доступно онлайн: <http://www.itu.int/rec/T-REC-Z.151/en> (дата обращения: 8 октября 2024 г.).
23. Аффлек, А.; Кришна, А.; Ачутан, Н. Р. Оптимальный выбор операционализаций для нефункциональных требований. В материалах Девятой Азиатско-Тихоокеанской конференции по концептуальному моделированию — том 143, APCCM'13, Аделаида, Австралия, 29 января – 1 февраля 2013 г.; стр. 69–78.
24. ДеВриз, Б.; Ченг, Б. Целеориентированное моделирование и анализ нефункциональных требований. В материалах 22-й Международной конференции ACM/IEEE 2019 года по языкам и системам проектирования на основе моделей (MODELS 2019), Мюнхен, Германия, 15–20 сентября 2019 г.; стр. 261–271. <https://doi.org/10.1109/MODELS.2019.00010>.
25. Гонсалвес, Дж.; Кришна, А. Разработка агентов на основе динамических нефункциональных требований и моделей. В материалах 24-й Австралийской конференции по программной инженерии (ASWEC 2015), Аделаида, Южная Австралия, 28 сентября – 1 октября 2015 г.; стр. 128–137. <https://doi.org/10.1109/ASWEC.2015.24>.
26. Ариф, М.; Мохаммад, К.; Садик, М. Метод анализа требований к системе, основанный на UML и NFR информационная система. Int. J. Inf. Technol. 2023, 15, 411–422. <https://doi.org/10.1007/s41870-022-01112-7>.

27. Мехта, Р.; Руис-Лопес, Т.; Чунг, Л.; Ногера, М. Выбор альтернатив с использованием зависимостей: подход NFR. В материалах 28-го ежегодного симпозиума ACM по прикладным вычислениям, SAC '13, Нью-Йорк, штат Нью-Йорк, США, 18–22 марта 2013 г.; стр. 1292–1297. <https://doi.org/10.1145/2480362.2480603>.
28. Пейшоту, М.М.; Сильва, К. Определение требований конфиденциальности с помощью языков целевого моделирования. В материалах XXXII Бразильского симпозиума по программной инженерии, SBES '18, Нью-Йорк, штат Нью-Йорк, США, 17–21 сентября 2018 г.; стр. 112–121. <https://doi.org/10.1145/3266237.3266270>.
29. Дермеваль, Д.; Кастро, Дж.; Сильва, К.; Пиментель, Дж.; Биттенкур, И.; Брито, П.; Элиас, Э.; Тенорио, Т.; Педро, А. Об использовании метамоделирования для связи требований и решений по архитектурному проектированию. В материалах 28-го ежегодного симпозиума ACM по прикладным вычислениям, SAC '13, Нью-Йорк, штат Нью-Йорк, США, 18–22 марта 2013 г.; стр. 1278–1283. <https://doi.org/10.1145/2480362.2480601>.
30. Вилела, Дж.; Кастро, Дж.; Мартинс, Л. Э. Г.; Горшек, Т.; Сильва, К. Определение требований безопасности с помощью языков GORE. В материалах XXXI Бразильского симпозиума по программной инженерии, SBES '17, Нью-Йорк, штат Нью-Йорк, США, 20–22 сентября 2017 г.; стр. 154–163. <https://doi.org/10.1145/3131151.3131175>.
31. Якобсон, Л.; Буч, Дж. Р. Г. Справочное руководство по унифицированному языку моделирования; Addison Wesley Longman, Inc.: Бостон, Массачусетс, США, 2021.
32. Буч, Г.; Рамбо, Дж.; Якобсон, И. Руководство пользователя унифицированного языка моделирования, 2-е изд.; Addison-Wesley: Бостон, Массачусетс, США; Pearson Education Inc.: Уэстфорд, Массачусетс, США, 2005.
33. Фуэнтес-Фернандес, Л.; Вальесильо-Морено, А. Введение в профили UML. UML-модель англ. 2004, 2, 72.
34. Саадатманд, М.; Тахвили, С. Нечёткий подход к поддержке принятия решений для анализа компромиссов нефункциональных требований на основе моделей. В материалах 12-й Международной конференции по информационным технологиям — Новые поколения, Лас-Вегас, Невада, США, 13–15 апреля 2015 г.; стр. 112–121. <https://doi.org/10.1109/ITNG.2015.24>.
35. Каур, Х.; Шарма, А. Мера моделирования нефункциональных требований с использованием расширенного варианта использования. В материалах Третьей международной конференции «Вычислительная техника для устойчивого глобального развития» (INDIACom), Нью-Дели, Индия, 16–18 марта 2016 г.; стр. 1101–1105.
36. Мартинес, Г. Г.; Карлио, А. Ф. Д.; Гомес, Л. Н. Модель обнаружения конфликтов и зависимостей в нефункциональных требованиях с использованием сценариев и вариантов использования. В материалах 45-й Латиноамериканской компьютерной конференции 2019 г. (CLEI 2019), Панама, 30 сентября – 4 октября 2019 г. <https://doi.org/10.1109/CLEI47609.2019.235051>.
37. Халик, Ф.; Батт, У.Х.; Хан, С.А. Создание нефункциональных требований к предметной области. Разработка линейки программных продуктов с использованием преобразований моделей. В материалах Международной конференции «Рубежи информационных технологий» 2017 г., FIT 2017, Исламабад, Пакистан, 18–20 декабря 2017 г.; Том 2017, стр. 41–45. <https://doi.org/10.1109/FIT.2017.00015>.
38. Махеш, Б. Алгоритмы машинного обучения — обзор. Int. J. Sci. Res. (IJSR) 2020, 9, 381–386.
39. Пуйанфар, С.; Садик, С.; Ян, Ю.; Тянь, Х.; Тао, Ю.; Рейес, М.П.; Шью, М.Л.; Чен, С.К.; Айенгар, С.С. Обзор глубокого обучения: Алгоритмы, методы и приложения. ACM Comput. Surv. (CSUR) 2018, 51, 1–36.
40. Маньини, Б.; Лавелли, А.; Магнолини, С. Сравнение подходов машинного обучения и глубокого обучения к задачам НЛП для итальянского языка. В материалах двенадцатой конференции по языковым ресурсам и оценке, Марсель, Франция, 11–16 мая 2020 г.; Кальцолари Н., Беше Ф., Блаш П., Шукри К., Сиери К., Деклерк Т., Годжи С., Исахара Х., Мегаард Б., Мариани Дж. и др., Ред.; стр. 2110–2119.
41. Рашван, А.; Орманджиева, О.; Витте, Р. Онтологическая классификация нефункциональных требований в спецификациях программного обеспечения: новый корпус и классификатор на основе SVM. Труды Международной конференции по компьютерному программному обеспечению и приложениям, Лос-Анджелес, Невада, США, 14 марта 2013 г.; стр. 381–386. <https://doi.org/10.1109/COMPSAC.2013.64>.
42. Каур, К.; Каур, П. BERT-CNN: Улучшение BERT для классификации требований с использованием CNN. Procedia Comput. Sci. 2023, стр. 218, 2604–2611. <https://doi.org/https://doi.org/10.1016/j.procs.2023.01.234>.
43. Рахман, М.А.; Хак, М.А.; Таухид, М.Н.А.; Сиддик, М.С. Классификация нефункциональных требований с использованием вариантов рекуррентных нейронных сетей для разработки качественного программного обеспечения. В материалах 3-го международного семинара ACM SIGSOFT по методам машинного обучения для оценки качества программного обеспечения, MaTeSQuE 2019, Нью-Йорк, США, 27 августа 2019 г.; стр. 25–30. <https://doi.org/10.1145/3340482.3342745>.
44. Яхья, А.Е.; Гарби, А.; Яфуз, В.М.; Аль-Дакм, А. Новая гибридная модель глубокого обучения для обнаружения и классификации нефункциональных требований проблем мобильных приложений. Электроника 2023, 12, 1258. <https://doi.org/10.3390/electronics12051258>.
45. Алашкар, А. М. Изучение общих черт, сопоставлений и взаимосвязей между нефункциональными требованиями с использованием машинного обучения. Научные вычислительные программы. 2022, 218, 102806. <https://doi.org/https://doi.org/10.1016/j.scico.2022.102806>.
46. Сингх, П.; Сингх, Д.; Шарма, А. Система правил для автоматизированной классификации нефункциональных требований из спецификаций требований. В материалах Международной конференции по достижениям в области вычислительной техники, коммуникаций и информатики (ICACCI), Джайпур, Индия, 21–24 сентября 2016 г.; стр. 620–626. <https://doi.org/10.1109/ICACCI.2016.7732115>.
47. Майриза, Д.; Зовги, Д.; Джерваси, В. Характеристика конфликтов и анализ нефункциональных требований: экспериментальный подход. В материалах конференции SoMeT 2013 — 12-й Международной конференции IEEE по методологиям, инструментам и методам интеллектуального программного обеспечения, Будапешт, Венгрия, 22–24 сентября 2013 г.; стр. 83–91. <https://doi.org/10.1109/SoMeT.2013.6645645>.
48. Чжан, Х.; Ван, Х. Анализ компромиссов для конфликтующих нефункциональных требований к программному обеспечению. IEEE Access 2019, 7, 156463–156475. <https://doi.org/10.1109/ACCESS.2019.2949218>.

49. Паукар, Л. Х. Г.; Бенкомо, Н. RE-STORM: Сопоставление проблемы принятия решений и компромисса нефункциональных требований с частично наблюдаемыми процессами принятия решений Маркова. В материалах 13-й Международной конференции по программной инженерии для адаптивных и самоуправляемых систем (SEAMS '18), Нью-Йорк, США, 28–29 мая 2018 г.; стр. 19–25. <https://doi.org/10.1145/3194133.3195537>.
50. Шах, У.; Патель, С.; Джинвала, Д. Онтологический подход к определению конфликтов между нефункциональными требованиями. В материалах 2-й Международной конференции по геоинформатике и анализу данных (ICGDA 2019), Нью-Йорк, США, 15–17 марта 2019 г.; стр. 145–149. <https://doi.org/10.1145/3318236.3318257>.
51. Лю, Ч. Л. (CDNFRE): Детектор конфликтов в развитии нефункциональных требований на основе онтологий. *Comput. Stand. Interfaces* 2016, 47, 62–76. <https://doi.org/https://doi.org/10.1016/j.csi.2016.03.002>.
52. Луангвирия, Т.; Конгкандра, Р. Автоматическое обнаружение конфликтов в нефункциональном анализе требований с использованием концептуального графа. *ECTI Trans. Comput. Inf. Technol.* 2023, 17, 82–94.
53. Алмейда, А.; Бенкомо, Н.; Батиста, Т.; Кавальканте, Э.; Дантас, Ф. Динамическое принятие решений на основе NFR для управления изменчивостью программного обеспечения и выбором конфигурации. В материалах 30-го ежегодного симпозиума ACM по прикладным вычислениям, SAC '15, Нью-Йорк, штат Нью-Йорк, США, 13–17 апреля 2015 г.; стр. 1376–1382. <https://doi.org/10.1145/2695664.2695875>.
54. Шах, Т.; Патель, С.В. Новый подход к заданию функциональных и нефункциональных требований с использованием RDS (схемы описания требований). *Procedia Comput. Sci.* 2016, 79, 852–860. <https://doi.org/https://doi.org/10.1016/j.procs.2016.03.083>.
55. Мензис, Т.; Кришна, Р.; Прайор, Д.; Репозиторий эмпирических данных по программной инженерии Promise 2015; Университет штата Северная Каролина, кафедра компьютерных наук: Роли, Северная Каролина, США, 2015. Доступно онлайн: <http://openscience.us/repo> (дата обращения : 8 октября 2024 г.).
56. Ван, Р.; Ли, Дж. Байесовский тест точности, полноты и меры F1 для сравнения двух моделей обработки естественного языка. В материалах 57-го ежегодного собрания Ассоциации компьютерной лингвистики, Флоренция, Италия, 28 июля – 2 августа 2019 г.; стр. 4135–4145.
57. Альджаллаби, Б.М.; Мансур, А. Улучшение подхода к анализу нефункциональных требований в среде Agile. В материалах Международной конференции по вычислительной технике, управлению, сетям, электронике и встраиваемым системам (ICCNEE) 2015 года, Хартум, Судан, 7–9 сентября 2015 г.; стр. 428–433. <https://doi.org/10.1109/ICCNEE.2015.7381407>.
58. Мухаммад, А.; Сиддик, А.; Мубашер, М.; Олдриш, А.; Навид, К. Н. Приоритизация нефункциональных требований в гибком процессе с использованием многокритериального анализа принятия решений. *IEEE Access* 2023, 11, 24631–24654. <https://doi.org/10.1109/ACCESS.2023.3253771>.
59. Аморндеттавин, М.; Сенивонгсе, Т. Нефункциональные шаблоны требований для гибкой разработки программного обеспечения. В материалах 3-й Международной конференции по программному обеспечению и электронному бизнесу (ICSEB '19), Токио, Япония, 9–11 декабря 2019 г.; стр. 66–74. <https://doi.org/10.1145/3374549.3374561>.
60. Братаас, Г.; Мартини, А.; Ханссен, Г.К.; Рэдер, Г. Гибкое выявление требований масштабируемости для открытых систем: пример из практики. *J. Сист. Программное обеспечение* 2021, 182, 111064. <https://doi.org/https://doi.org/10.1016/j.jss.2021.111064>.
61. Хаттер, К.; Калия, А. Интеграция нефункциональных требований в архитектуру, основанную на моделях. В материалах Пятой международной конференции «Достижения в новейших технологиях в области связи и вычислений» (ARTCom 2013), Бангалор, Индия, 20–21 сентября 2013 г.; стр. 359–364. <https://doi.org/10.1049/cp.2013.2213>.
62. Мериджи, К.; Аль-Сарайрех, К.Т.; Абран, А.; Трудел, С. Требования к безопасности системы: структура для раннего выявления, спецификации и измерения соответствующих требований к программному обеспечению. *Comput. Stand. Interfaces* 2019, 66, 103346. <https://doi.org/https://doi.org/10.1016/j.csi.2019.04.005>.
63. Бербак, Р. Методология программной инженерии: Водосброс. Кандидатская диссертация, Стэнфордский университет, кафедра компьютерных наук. Наука: Стэнфорд, Калифорния, США, 1999.
64. Отеро, К. Проектирование программного обеспечения: теория и практика; CRC Press: Бока-Ратон, Флорида, США, 2012.
65. IEEE Std 830-1998; Рекомендуемая практика IEEE для спецификаций требований к программному обеспечению; IEEE: Пискатауэй, Нью-Джерси, США, 1998; стр. 1–40. <https://doi.org/10.1109/IEEESTD.1998.88286>.
66. Вернер, К.; Ли, З.С.; Лоулинд, Д.; Элажари, О.; Эрнст, Н.; Дамиан, Д. Непрерывное управление нефинансовыми показателями: возможности и проблемы на практике. *IEEE Trans. Software Eng.* 2022, 48, 2629–2642. <https://doi.org/10.1109/TSE.2021.3066330>.
67. Гопичанд, М.; Рао, А.А. Четырехуровневый подход к анализу нефункциональных требований. arXiv 2012, arXiv:1201.6141.
68. Кассаб, М.; Данева, М.; Орманджиева, О. Ранняя количественная оценка нефункциональных требований; Центр телематики и информационных технологий (СТИТ), Университет Твенте: Энскеде, Нидерланды, 2007. Доступно онлайн: <https://research.utwente.nl/en/publications/early-quantitative-assessment-of-non-functional-requirements> (дата обращения: 8 октября 2020 г.).
69. Шарма, В.С.; Рамнани, Р.Р.; Сенгупта, С. Структура для выявления и анализа нефункциональных требований из текста. В материалах 4-го международного семинара по требованиям и архитектуре Twin Peaks, TwinPeaks 2014, Нью-Йорк, США, 1 июня 2014 г.; стр. 1–8. <https://doi.org/10.1145/2593861.2593862>.
70. Мацумото, Й.; Ширази, С.; Ониши, А. Метод проверки нефункциональных требований. *Procedia Comput. Sci.* 2017, стр. 112, 157–166. <https://doi.org/https://doi.org/10.1016/j.procs.2017.08.006>.
71. Матусси, А.; Лало, Р. Обзор нефункциональных требований в процессе разработки программного обеспечения. Исследовательский отчет TR-LACL-2008-7, LACL. 2008. Доступно онлайн: <https://hal.science/hal-01224656v1> (дата обращения: 8 октября 2024 г.).

72. Экхардт, Дж.; Фогельсанг, А.; Фернандес, Д.М. Действительно ли «нефункциональные» требования нефункциональны? Исследование нефункциональных требований на практике. В материалах 38-й Международной конференции по программной инженерии, ICSE '16, Нью-Йорк, США, 14–22 мая 2016 г.; стр. 832–842. <https://doi.org/10.1145/2884781.2884788>.
73. Цай, К. И. Нефункциональные вычисления: к более научному подходу к нефункциональным требованиям. В материалах конференции «Программное обеспечение и приложения» , 2007, COMPSAC 2007, 31-й ежегодный международный конгресс, Пекин, Китай, 23–27 июля 2007 г.; том 2, стр. 493–494. <https://doi.org/10.1109/COMPSAC.2007.156>.
74. Роза, Н.С.; Кунья, П.Р.Ф. Подход к обоснованию и уточнению нефункциональных требований. J. Braz. Comp. Soc. 2004, 10, 59–81.
75. Абдалла, А.Е.; Боуэн, Дж.П.; Ниссанке, Н. Формальные методы для систем, критически важных для безопасности. В: Надежные вычислительные системы: парадигмы, проблемы производительности и приложения, часть I: Модели и парадигмы; Серия Wiley по параллельным и распределенным вычислениям; под ред. Диаба, Х.Б., Зомайи, А.Ю.; John Wiley & Sons: Чичестер, Великобритания, 2005; том 9.

Отказ от ответственности/Примечание издателя: заявления, мнения и данные, содержащиеся во всех публикациях, принадлежат исключительно отдельным авторам и соавторам, а не MDPI и/или редакторам. MDPI и/или редакторы не несут ответственности за любой ущерб, причиненный людям или имуществу в результате использования любых идей, методов, инструкций или продуктов, упомянутых в материалах.